

6 Pakete, Generics und Java-Bibliothek

6.1 Pakete

6.2 Generische Typen

6.3 Bibliothek

6 Pakete, Generics und Java-Bibliothek

6.1 Pakete

6.2 Generische Typen

6.3 Bibliothek

Paket (*package*)

- ein Paket ist eine Sammlung zusammengehöriger Klassen/Schnittstellen
- Pakete können hierarchisch strukturiert werden

Zweck

- bessere Kontrolle der Zugriffsrechte (wer darf auf was zugreifen)
- Vermeidung von Namenskonflikten

Beispiele für Pakete in der Java-Bibliothek

<code>java.lang</code>	Object, String, Integer, Math, IO
<code>java.io</code>	File, InputStream, OutputStream, Reader, Writer
<code>java.util</code>	List, ArrayList, Map, TreeMap, Random
<code>java.awt</code>	Button, CheckBox, Frame, List, Color, Cursor
<code>java.awt.geom</code>	Rectangle2D, Ellipse2D, Line2D, Point2D
<code>javax.swing</code>	JButton, JTextArea, JTextField, JPanel, JFrame

Der Paketname wird in der ersten Zeile einer Java-Datei angegeben

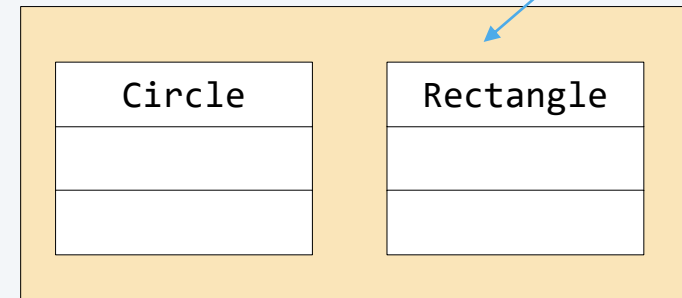
Beispiel: Paket shapes enthält die Klassen Circle und Rectangle

Circle.java

```
package shapes;  
class Circle {  
    ...  
}
```

Rectangle.java

```
package shapes;  
class Rectangle {  
    ...  
}
```



Wenn package fehlt, gehören die Klassen zu einem namenlosen Paket

Pakete sind Sichtbarkeitsgrenzen

- was in einem Paket deklariert ist, ist in anderen Paketen unsichtbar
- in verschiedenen Paketen können gleiche Namen verwendet werden

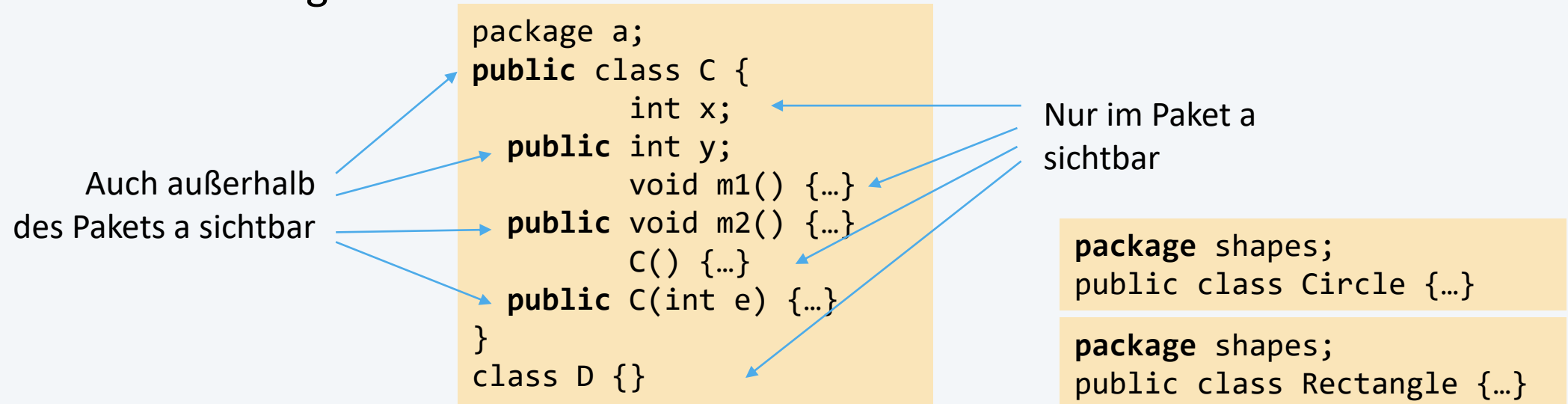
```
package a;  
class C {}  
class D {}
```

Gleicher Name C in
a und b möglich

```
package b;  
class C {  
    D d;  
}
```

D ist hier nicht
sichtbar
(Compilerfehler!)

Mit **public** können Klassen, Methoden und Datenkomponenten für andere Pakete sichtbar gemacht werden



Öffentliche Klassennamen können in anderen Paketen importiert werden

```
package app;  
import shapes.Circle;  
  
class App {  
    Circle c;  
}
```

Import einer Klasse

```
package app;  
import shapes.*;  
class App {  
    Circle c;  
    Rectangle r;  
}
```

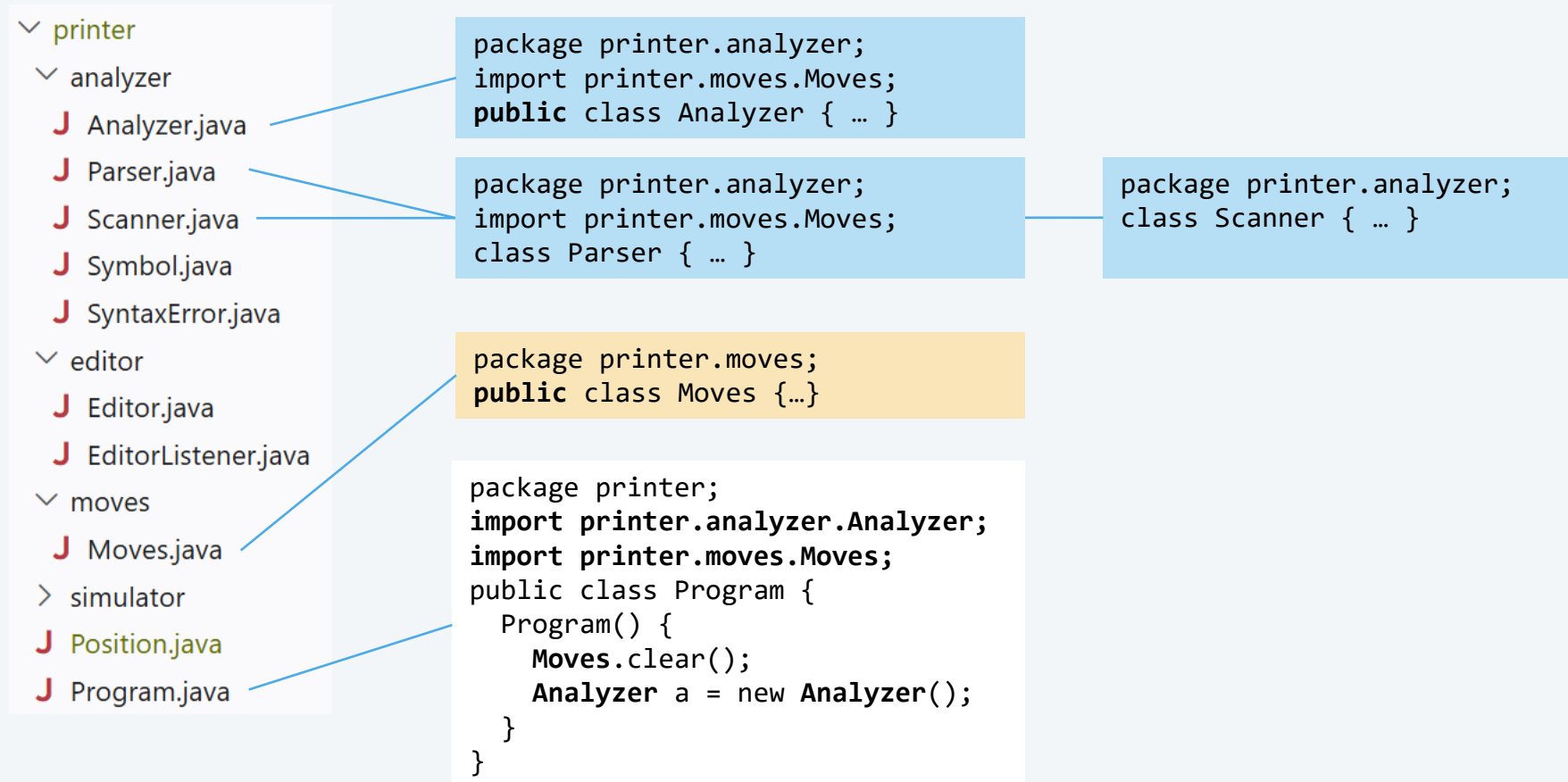
Import aller public-
Klassen eines Pakets

```
package app;  
class App {  
    shapes.Circle c1;  
    graphics.Circle c2;  
}
```

Qualifikation mit dem
Paketnamen

Pakete werden auf Verzeichnisse abgebildet, Klassen auf Dateien

Beispiel: Klassen für 3D-Drucker-Simulation



Sichtbarkeit von Klassen

- `public` Klassen können auch außerhalb des Pakets benutzt werden
- nicht `public` Klassen sind nur innerhalb des Pakets sichtbar

Sichtbarkeit von Datenkomponenten und Methoden

- wird durch `public`, `protected` oder `private` definiert
- ohne Angabe eines Sichtbarkeitsattributs gilt der Sichtbarkeitsbereich Paket (*package visibility*)

Datenk./Methoden sind zugreifbar in ...	... wenn für Datenk./Methoden gilt:			
	public	protected	(<i>package</i>)	private
... selber Klasse	ja	ja	ja	ja
... anderer Klasse in gleichem Paket	ja	ja	ja	nein
... abgeleit. Klasse in anderem Paket	ja	ja	nein	nein
... anderer Klasse in anderem Paket	ja	nein	nein	nein

6 Pakete, Generics und Java-Bibliothek

6.1 Pakete

6.2 Generische Typen

6.3 Bibliothek

Motivation: Problem bei Verwendung allgemeiner Objektsammlungen

- Zugriff erfordert Typumwandlung (type cast)
- Keine Typsicherheit durch Compiler

```
ArrayList shapeList = new ArrayList();

for (int i = 0; i < shapeList.size(); i++) {
    Shape s = (Shape)shapeList.get(i);
    s.draw();
}
```

statischer Typ Object

ArrayList
elements: Object [] n: int
ArrayList() add(e: Object) get(i: int): Object remove(e: Object) size(): int

Beispiel

```
ArrayList shapeList = new ArrayList();
shapeList.add(new Circle(...));
shapeList.add(new Line(...));
shapeList.add(new Person(...));
```

der statische Typ Object
erlaubt hier jedes Java-Objekt

```
Shape s = (Shape)shapeList.get(0)
```

```
Shape s = (Shape)shapeList.get(2)
```

ClassCastException

Generische Klassen und Interfaces bieten die Möglichkeit, einen Behälter (Container) so einzuschränken, dass er nur Objekte einer bestimmten Klasse bzw. deren Unterklassen aufnehmen kann

- Klassen und Interfaces mit Typparameter
- Typparameter sind Platzhalter für Typen

Beispiel: Generische Klasse ArrayList mit Typparameter E

- Typparameter E ist Platzhalter für konkreten Typ

```
public class ArrayList<E> ... {  
    private E[] elements;  
    private int n;  
  
    public      ArrayList() { ... }  
    public void add(E e) { ... }  
    public      E get(int i) { ... }  
    public void remove(E e) { ... }  
    public int size() { ... }  
}
```

```
public class ArrayList ... {  
    private Object[] elements;  
    private int n;  
  
    public      ArrayList() { ... }  
    public void add(Object e) { ... }  
    public Object get(int i) { ... }  
    public void remove(Object e) { ... }  
    public int size() { ... }  
}
```

Klasse ArrayList (Kapitel 3)

Beispiel: Generische Typen und Felder

- der Compiler prüft, ob Typparameter (E) korrekt verwendet wird
- die Typparameter werden zur Laufzeit **nicht gespeichert** (*type erasure*)
- es ist daher nicht möglich, Felder mit Typparametern zu erzeugen, z.B. `this.elements = new E[16]`

```
public class ArrayList<E> ... {  
    private E[] elements;  
    private int n;  
  
    public ArrayList() { ...  
        this.elements = (E[]) new Object[16];  
        this.n = 0;  
    }  
    ...  
}
```

Zuweisung erfordert Typzusicherung

Parametrisierung mit konkretem Typ

```
ArrayList<Shape> shapeList = new ArrayList<Shape>();
```

```
ArrayList<String> nameList = new ArrayList<String>();
```

```
public class ArrayList<Shape> ... {  
    Shape[] elements;  
    ...  
    void add(Shape e) { ... }  
    Shape get(int i) { ... }  
}
```

```
public class ArrayList<String> ... {  
    void add(String e) { ... }  
    String get(int i) { ... }  
}
```

Typsicheres Arbeiten durch konkrete Typen für Typparameter

```
shapeList.add(new Circle(...));  
shapeList.add(new Line(...));
```

```
shapeList.add(new Person(...));
```

Nicht möglich (prüft Compiler!)

```
for (int i = 0; i < shapeList.size(); i++) {  
    Shape s = shapeList.get(i);  
    s.draw();  
}
```

```
nameList.add("Alice");
```

```
nameList.add(new Circle(...));
```

Nicht möglich (prüft Compiler!)

Ohne Parametrisierung kann/soll generische Klasse nicht verwendet werden

```
ArrayList shapeList;
```

```
found raw type: ArrayList  
missing type arguments for generic class ArrayList<T>
```

```
new ArrayList();
```

```
found raw type: ArrayList  
missing type arguments for generic class ArrayList<T>
```

Beim Erzeugen von Objekten dürfen die aktuellen Typparameter weggelassen werden, wenn sie aus dem Kontext ableitbar sind

```
ArrayList<Shape> shapeList = new ArrayList<>();
```

Aus **Sicht des Compilers** sind `ArrayList<Shape>`, `ArrayList<String>` und `ArrayList<Person>` **verschiedene Typen**

- es reicht jedoch genau eine Implementierung (*raw type*)

Bei Ableitung eines generischen Typs muss der Typparameter festgelegt werden

- Möglichkeit 1: Abgeleiteter Typ ist ebenfalls generischer Typ

```
public class ArrayList<E> implements List<E> {  
    public void add(E e) { ... }  
    public      E get(int i) { ... }  
    ...  
}
```

```
public interface List<E> {  
    public void add(E e);  
    public      E get(int i);  
    ...  
}
```

- Möglichkeit 2: Abgeleiteter Typ ist kein generischer Typ

```
public class StringArrayList implements List<String> {  
    public void add(String e) { ... }  
    public String get(int i) { ... }  
    ...  
}
```

6 Pakete, Generics und Java-Bibliothek

6.1 Pakete

6.2 Generische Typen

6.3 Bibliothek

- darunter verstehen wir eine Menge von Klassen und Interfaces, zusammen mit ihrem Beziehungsgeflecht, z.B. Vererbungsbeziehungen
- bei einfacher Vererbung durch eine oder mehrere Klassenhierarchien repräsentiert
- bei mehrfacher Vererbung durch einen oder mehrere azyklische, gerichtete Graphen repräsentiert

Bedeutung von Klassenbibliotheken

- sie bilden eine wichtige Basis für den Software-Entwicklungsprozess
- die Klassenbibliothek „erweitert“ die Programmiersprache
- Sprache tritt u.U. hinter die Klassenbibliothek zurück

Umfang in Java 25: mehrere tausend Klassen/Schnittstellen

Beispiele

- Wrapper-Klassen für Wertdatentypen (`java.lang`)
- Klassen für Datenstrukturen (`java.util`)
- Betriebssystemklassen (`java.lang`, `java.io`)
- Klassen für grafische Objekte (`java.awt`, `java.awt.geom`)
- Benutzerschnittstellenklassen (`javax.swing`)

- Wertdatentypen `int`, `float`, `boolean` etc. sind nicht mit dem Referenzdatentyp `Object` kompatibel
- Verpacken in Wrapper-Klassen notwendig

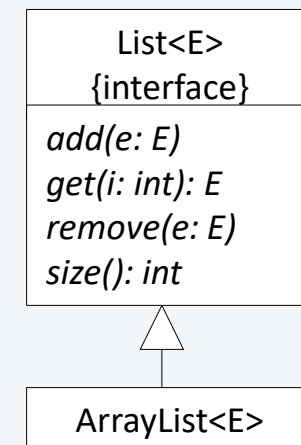
```
int i = 3;
Integer b = Integer.valueOf(i);
Object c = b;
int j = b.intValue();
```

Stil: `new Integer(i)` ist möglich
aber veraltet (*deprecated*)

- Objekte dieser Klassen "umhüllen" Werte von Wertdatentypen

```
List<Integer> nums = new ArrayList<Integer>();
for (int i = 1; i <= 100; i++) {
    nums.add(Integer.valueOf(i));
}

int sum = 0;
for (int i = 0; i < nums.size(); i++) {
    Integer iObj = nums.get(i);
    sum += iObj.intValue();
}
```



Klasse Integer (Auszug):

```
package java.lang;
public class Integer extends Number {

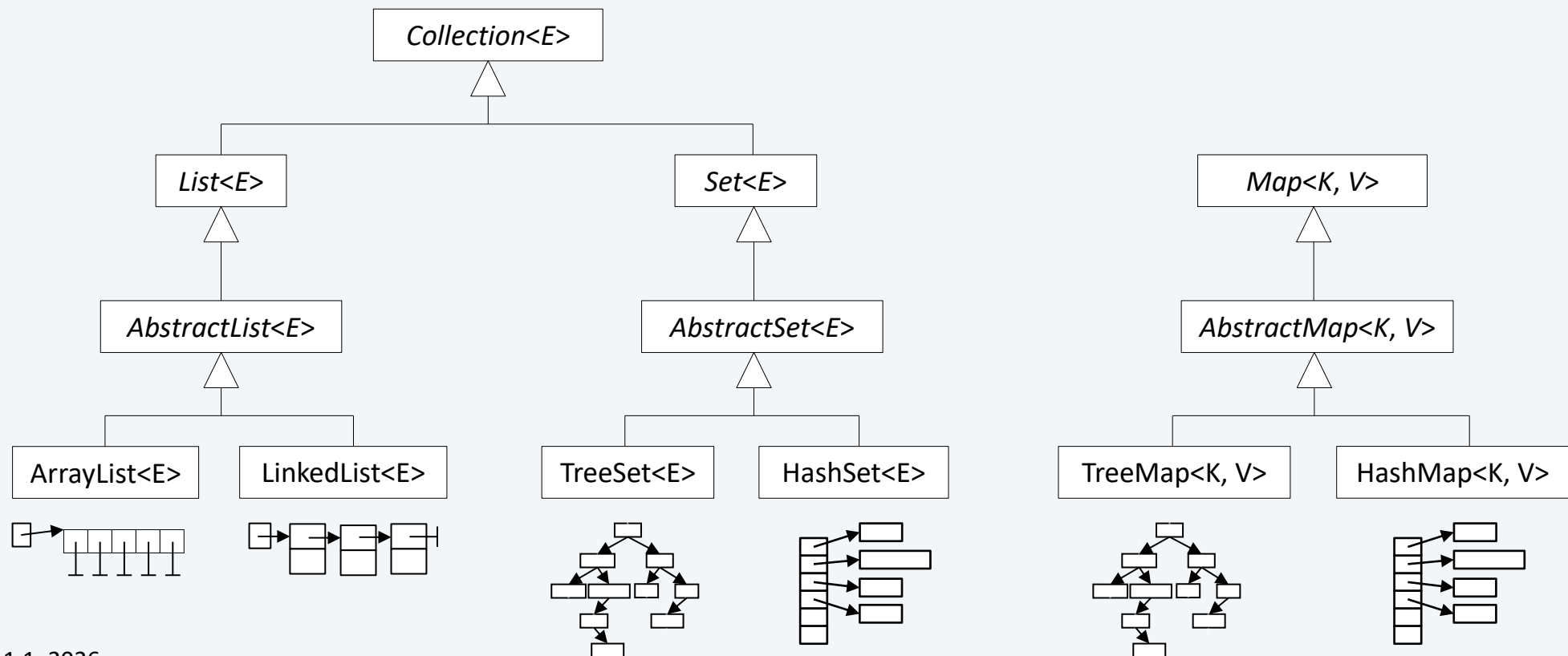
    public static final int MIN_VALUE = 0x80000000;
    public static final int MAX_VALUE = 0x7fffffff;
    ...
    public static      int parseInt(String s) throws NumberFormatException {...}
    public static Integer valueOf(int i) {...}
    ...
    private final int value;

    public  String toString() { ... }
    public      int hashCode() { return value; }
    public boolean equals(Object obj) { ... }
    public      int intValue() { return value; }
    ...
}
```

Weitere Klassen: Byte, Short, Boolean, Character, Float, Double

- Klassen für Objekte zur Aufnahme anderer Objekte (Behälterklassen, engl. *container classes* oder *collection classes*)
- typische Operationen: Einfügen, Löschen, Suchen und Sortieren

Interfaces und Klassen in Paket `java.util` (Auszug, vereinfacht):



- bilden Funktionen der Hardware und des Betriebssystems mit Objekten nach
- kapseln Details des Betriebssystems und verbessern so die Portabilität darauf aufbauender Programme

Java-Beispiele

- Prozesse: Process, Thread , Semaphore
- Netze: Socket, InetAddress
- Dateien: File, InputStream

Die Klasse File beschreibt einen Pfad im Dateisystem

```
File a = new File("a.txt");  
File temp = new File("C:/temp");
```

Methoden (Auszug)

boolean	exists()	Existiert dieser Pfad?
boolean	isFile()	Ist es eine Datei?
boolean	isDirectory()	Ist es ein Verzeichnis?
String	getName()	Name der Datei/des Verzeichnisses
String	getAbsolutePath()	Absoluter Pfad
long	length()	Größe der Datei in Bytes
File[]	listFiles()	Liste der Dateien im Verzeichnis
boolean	canRead()	Darf Datei/Verzeichnis gelesen werden?
boolean	canWrite()	Darf Datei/Verzeichnis geschrieben werden?

- ein Bytestrom ist eine Folge von Bytes
- alle Byteströme sind von den abstrakten Klassen `InputStream` und `OutputStream` abgeleitet

<i>InputStream</i>
<code>int read()</code> <code>int read(byte[] b)</code> <code>int available()</code> <code>void close()</code>

<i>OutputStream</i>
<code>void write(int b)</code> <code>void write(byte[] b)</code> <code>void flush()</code> <code>void close()</code>

Alle Methoden können
`IOException` auslösen

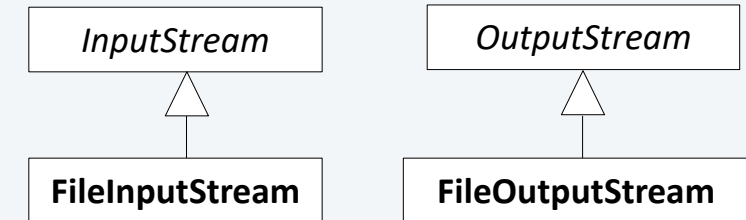
Beispiel: Kopiere die Folge von Bytes von in nach out

```
void copy(InputStream in, OutputStream out) throws IOException {  
    int b = in.read();  
    while (b != -1) {  
        out.write(b);  
        b = in.read();  
    }  
}
```

Methoden `read/write` können
`IOException` auslösen

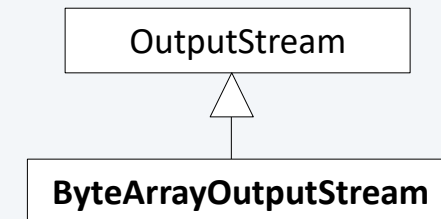
Beispiel: Kopiere Inhalt der Datei a.dat nach b.dat

```
InputStream in = null; OutputStream out = null;
try {
    in = new FileInputStream("a.dat");
    out = new FileOutputStream("b.dat");
    copy(in, out);
}
catch(FileNotFoundException e) { ... }
catch (IOException e) { ... }
finally { /* close in and out */ }
```



Beispiel: Kopiere Inhalt der Datei a.dat in Feld data

```
InputStream in = null; ByteArrayOutputStream out;
out = new ByteArrayOutputStream();
try {
    in = new FileInputStream("a.dat");
    copy(in, out);
}... // wie oben
byte[] data = out.toByteArray();
```

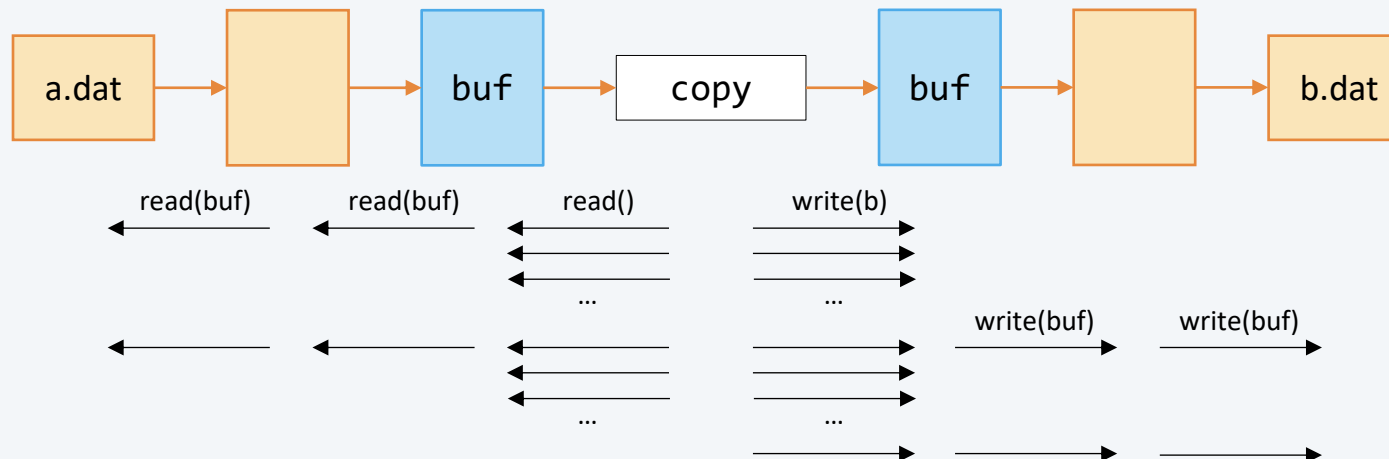


- viele einzelne Lese-/Schreibzugriffe auf Datei sind langsam
- Lösung durch Puffer zwischen Datei und Lese-/Schreibeoperation

Beispiel: Kopiere Inhalt der Datei a.dat nach b.dat

```
InputStream in = null; OutputStream out = null;  
try {  
    in = new BufferedInputStream(new FileInputStream("a.dat"));  
    out = new BufferedOutputStream(new FileOutputStream("b.dat"));  
    copy(in, out);  
}... // wie auf vorheriger Seite
```

Ablauf:



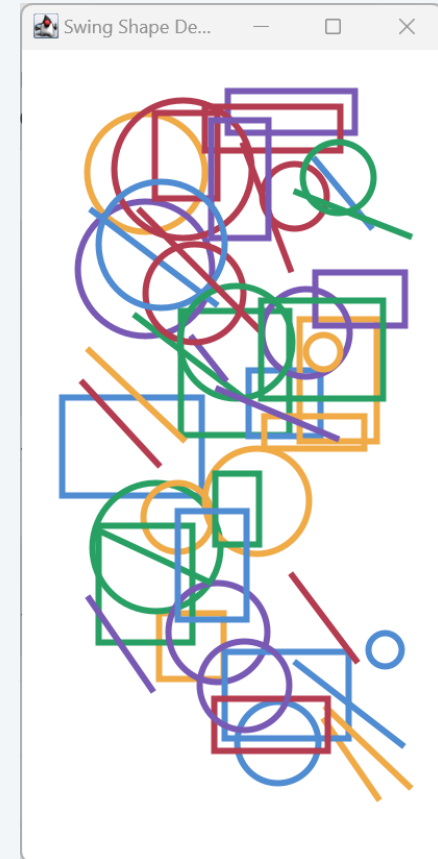
Dienen der Bereitstellung von Eigenschaften und Operationen elementarer und spezieller Grafikobjekte wie Linien, Rechtecke etc.

Beispiel:

```
import java.awt.Shape;
import java.awt.geom.Rectangle2D;
import java.awt.geom.Ellipse2D;

Shape[] figures = new Shape[count];
...
figures[7] = new Rectangle2D.Double(x, y, w, h);
figures[8] = new Ellipse2D.Double(x, y, w, w);
...

protected void paintComponent(Graphics g) {
    super.paintComponent(g);
    Graphics2D g2 = (Graphics2D) g.create();
    for (int i = 0; i < figures.length; i++) {
        g2.setColor(colors[i]);
        g2.draw(figures[i]);
    }
}
```

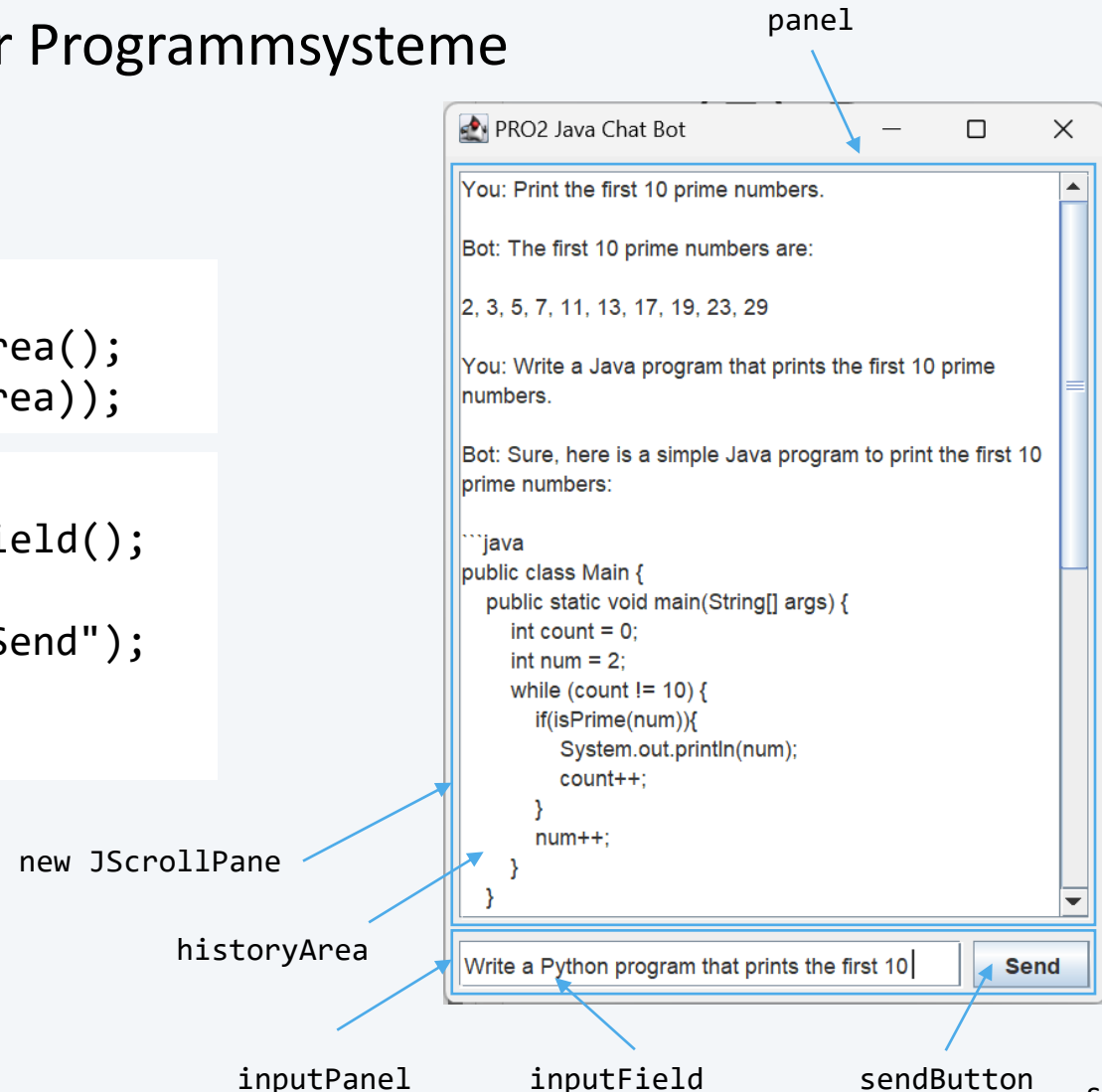


Dienen der Bereitstellung von Objekten zur Konstruktion der Benutzungsschnittstelle interaktiver Programmsysteme
(*graphical user interface*, GUI)

Beispiel (vereinfacht):

```
JPanel panel = new JPanel();  
JTextArea historyArea = new JTextArea();  
panel.add(new JScrollPane(historyArea));
```

```
JPanel inputPanel = new JPanel();  
JTextField inputField = new JTextField();  
inputPanel.add(inputField);  
JButton sendButton = new JButton("Send");  
inputPanel.add(sendButton);  
panel.add(inputPanel);
```



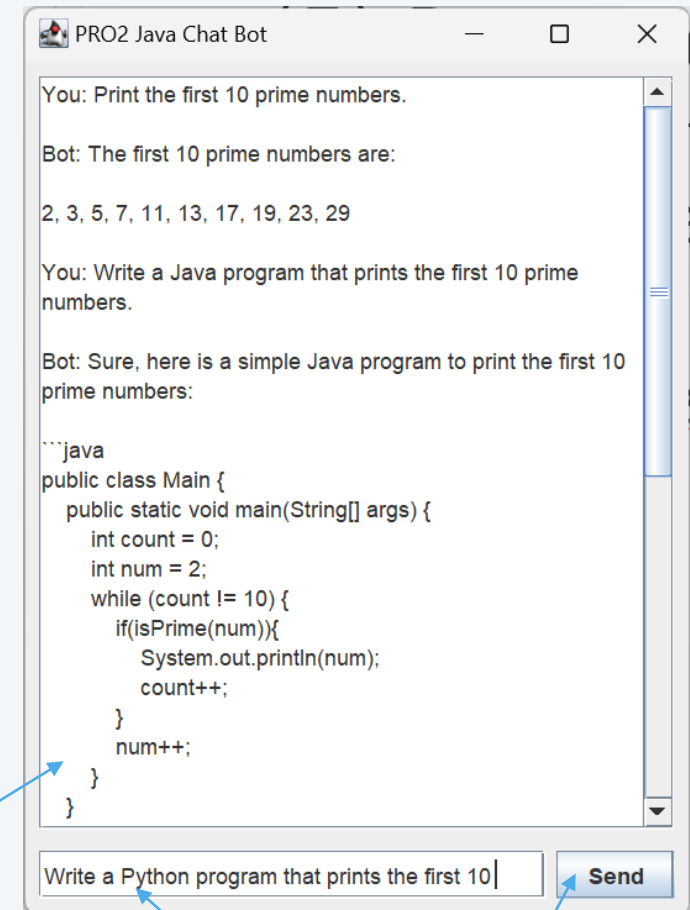
Beispiel: „Send“

```
class Chat implements ActionListener {
```

```
    sendButton.addActionListener(this);
```

```
    public void actionPerformed(ActionEvent e) {  
        String prompt = inputField.getText();  
        historyArea.append("You:" + prompt + "\n");  
        inputField.setText("");  
        inputField.setEnabled(false);  
        sendButton.setEnabled(false);  
  
        String message = llm.generateAnswer(prompt);  
  
        historyArea.append("Bot:" + message + "\n");  
        inputField.setEnabled(true);  
        sendButton.setEnabled(true);  
    }
```

historyArea



inputField

sendButton

6 Pakete, Generics und Java-Bibliothek

6.1 Pakete

6.2 Generische Typen

6.3 Bibliothek